

Part 4C: Phase 1 Linux

Admin Routes, Invite Codes, Deployment & Alpha-Ready Checklist

Developer

Daniel Boone

GitHub

github.com/dboone323/HavenWorld

Game URL

<https://havenworld.pages.dev>

API Base

[https://\[your-oracle-ip\].nip.io](https://[your-oracle-ip].nip.io)

Date

September 18, 2026

Series

Part 4C of 4C (Final Document)

📘 Platform Tag Legend

All steps in this document are tagged by where they are performed:

MAC Written and committed on the **MacBook Pro** (local dev machine).

LINUX

Executed directly on the **Oracle Cloud ARM VM** via SSH.

Table of Contents

1. Introduction — What This Document Covers
2. Section 12 — Admin Routes (MAC) — User Management, Reports, Chat Log, Stats
3. Section 13 — Invite Code Generation and Distribution (LINUX)
4. Section 14 — Production Deployment Workflow (MAC → LINUX via GitHub Actions)
5. Section 15 — Ongoing Maintenance and Monitoring (LINUX)
6. Section 16 — Final Alpha-Ready Gate Checklist (60+ items)
7. Section 17 — Phase 1 → Phase 2 Roadmap Preview
8. Appendix — Completed 4-Part Build Guide Summary Table

00 Introduction

Parts 4A and 4B established the server foundation, database schema, authentication system, RoomManager, ModerationService, and all Socket.io event handlers. This final document completes the Phase 1 Linux work and brings HavenWorld to alpha-ready status.

Section	Topic	Tag
12	Admin routes — user management, report review, invite codes, chat log access, server stats	MAC
13	Invite code generation CLI script and full alpha distribution workflow	LINUX

Section	Topic	Tag
14	Complete production deployment process — Mac to Oracle VM via GitHub Actions	Both
15	Ongoing server maintenance and monitoring commands for the alpha period	LINUX
16	Final alpha-ready gate checklist — every item must be checked before handing out invite codes	Both
17	Phase 1 → Phase 2 roadmap preview — what to build after alpha testers are playing	—

Do Not Skip Section 16

The alpha-ready gate checklist in Section 16 is not optional. Every item must be checked off before any invite code is distributed. A broken game given to alpha testers creates a negative first impression that is very difficult to recover from.

12 Admin Routes MAC

All admin endpoints require both `requireAuth` AND `requireRole(['ADMIN', 'MODERATOR'])` middleware. These endpoints are how Daniel and any trusted moderators manage the alpha — viewing users, acting on reports, reviewing chat history, and generating invite codes via the API.

12.1 Create `src/routes/admin.ts` MAC

Create `apps/server/src/routes/admin.ts` with the complete implementation below. This single file contains all admin-scoped routes organized into five groups: invite

code management, user management, report management, chat log access, and server stats.

✓ Route Access Summary

Routes marked **ADMIN only** use a nested `requireRole(['ADMIN'])` call and are not accessible to MODERATOR-role users. All other routes under the router-level `requireRole(['ADMIN', 'MODERATOR'])` are accessible to both roles.

File: `apps/server/src/routes/admin.ts`

```
import { Router } from 'express'; import crypto from 'crypto'; import { z } from 'zod'; import { requireAuth, requireRole, AuthRequest } from '../middleware/auth'; import { prisma } from '../prisma'; const router = Router(); // All admin routes require authentication + ADMIN or MODERATOR role router.use(requireAuth); router.use(requireRole(['ADMIN', 'MODERATOR'])); //

// INVITE CODE MANAGEMENT (ADMIN only) //

const generateInviteSchema = z.object({ count: z.number().int().min(1).max(100).default(25), expiryDays: z.number().int().min(1).max(365).default(30), }); // POST /api/admin/invites/generate – create a batch of invite codes router.post('/invites/generate', requireRole(['ADMIN']), async (req: AuthRequest, res) => { const parsed = generateInviteSchema.safeParse(req.body); if (!parsed.success) { return res.status(400).json({ error: 'Invalid input.', fields: parsed.error.flatten().fieldErrors }); } const { count, expiryDays } = parsed.data; const expiresAt = new Date(Date.now() + expiryDays * 86400000); const codes = Array.from({ length: count }, () => crypto.randomBytes(4).toString('hex').toUpperCase()); await prisma.inviteCode.createMany({ data: codes.map((code) => ({ code, createdBy: req.user!.userId, expiresAt, isActive: true, })), skipDuplicates: true, }); return res.status(201).json({ message: `Generated ${codes.length} invite codes expiring in ${expiryDays} days.`, codes, expiresAt, }); }); // GET /api/admin/invites – list all invite codes with usage status router.get('/invites', requireRole(['ADMIN']), async (_req, res) => { const invites = await prisma.inviteCode.findMany({ include: { usedBy: { select: { username: true } }, }, orderBy: { createdAt: 'desc' }, }); return res.json(invites); }); // DELETE /api/admin/invites/:id – deactivate an unused invite code router.delete('/invites/:id', requireRole(['ADMIN']), async (req, res) => { await prisma.inviteCode.update({ where: { id: req.params.id }, data: { isActive: false }, }); return res.json({ message: 'Invite code deactivated.' }); }); //

// USER MANAGEMENT //

// GET /api/admin/users – paginated user list with optional search router.get('/users', async (req, res) => { const page =
```

```

parseInt(req.query.page as string ?? '1', 10);    const limit =
parseInt(req.query.limit as string ?? '50', 10);    const search =
req.query.search as string | undefined;    const where = search    ? { OR: [
{ username: { contains: search, mode: 'insensitive' as const } },
{ email:    { contains: search, mode: 'insensitive' as const } },    ]
: {}];    const [users, total] = await
Promise.all([    prisma.user.findMany({        where,        select: {
id: true, username: true, email: true,        role: true, status: true,
emailVerified: true,        createdAt: true, lastLoginAt: true, mutedUntil:
true,    },        orderBy: { createdAt: 'desc' },        skip: (page - 1) *
limit,        take: limit,    }),    prisma.user.count({ where }),    ]);
return res.json({ users, total, page, pages: Math.ceil(total / limit) }); });
// GET /api/admin/users/:id – single user detail with inventory count
router.get('/users/:id', async (req, res) => {    const user = await
prisma.user.findUnique({        where: { id: req.params.id },        include: {
avatar: true,        _count: { select: { inventory: true, sentMessages: true,
sentReports: true } },    },    });    if (!user) return
res.status(404).json({ error: 'User not found.' });    return
res.json(user); });    const muteSchema = z.object({    hours:
z.number().int().min(1).max(720).default(24),    reason:
z.string().max(200).optional(),    });    // POST /api/admin/users/:id/mute – mute
a player for N hours router.post('/users/:id/mute', async (req: AuthRequest,
res) => {    const parsed = muteSchema.safeParse(req.body);    if (!
parsed.success) {        return res.status(400).json({ error: 'Invalid mute
data.' });    }    const mutedUntil = new Date(Date.now() + parsed.data.hours *
3600000);    await prisma.user.update({        where: { id: req.params.id },
data: { status: 'MUTED', mutedUntil },    });    console.log(`[Admin] $
{req.user!.username} muted user ${req.params.id} for ${parsed.data.hours}h`);
return res.json({ message: `User muted for ${parsed.data.hours} hours.`,
mutedUntil }); });    // POST /api/admin/users/:id/ban – permanently ban a
player router.post('/users/:id/ban', async (req: AuthRequest, res) =>
{    await prisma.user.update({        where: { id: req.params.id },        data:
{ status: 'BANNED', mutedUntil: null },    });    console.log(`[Admin] $
{req.user!.username} banned user ${req.params.id}`);    return
res.json({ message: 'User has been banned.' }); });    // POST
/api/admin/users/:id/unban – restore a muted or banned player
router.post('/users/:id/unban', async (req: AuthRequest, res) => {    await
prisma.user.update({        where: { id: req.params.id },        data: { status:
'ACTIVE', mutedUntil: null },    });    console.log(`[Admin] $
{req.user!.username} restored user ${req.params.id}`);    return
res.json({ message: 'User account restored to active.' }); });    // POST
/api/admin/users/:id/promote – promote a player to MODERATOR (ADMIN only)
router.post('/users/:id/promote', requireRole(['ADMIN']), async (req:
AuthRequest, res) => {    await prisma.user.update({        where: { id:
req.params.id },        data: { role: 'MODERATOR' },    });    return
res.json({ message: 'User promoted to Moderator.' }); });    //
// REPORT
MANAGEMENT //
// GET /api/admin/reports – list reports with optional status filter
router.get('/reports', async (req, res) => {    const status =
req.query.status as string | undefined;    const page =
parseInt(req.query.page as string ?? '1', 10);    const limit = 25;    const
where = status ? { status: status as any } : { status: 'OPEN' as any };
const [reports, total] = await Promise.all([    prisma.report.findMany({
where,        include: { reporter:        { select: { id: true, username:
true } },        reportedUser: { select: { id: true, username: true, status:
true } },    },        orderBy: { createdAt: 'desc' },        skip: (page -
1) * limit,        take: limit,    }),    prisma.report.count({ where }),    ]);
return res.json({ reports, total,
page, pages: Math.ceil(total / limit) }); });    // PATCH

```

```

/api/admin/reports/:id – update report status and add moderator notes
router.patch('/reports/:id', async (req: AuthRequest, res) => {  const
{ status, moderatorNotes } = req.body;  const validStatuses = ['OPEN',
'REVIEWED', 'RESOLVED', 'DISMISSED'];  if (!validStatuses.includes(status))
{  return res.status(400).json({ error: 'Invalid status value.' });  }
const updated = await prisma.report.update({  where: { id:
req.params.id },  data: {  status,  moderatorNotes:
moderatorNotes ?? undefined,  resolvedAt: ['RESOLVED',
'DISMISSED'].includes(status) ? new Date() : undefined,  },  });  return
res.json(updated); });  //
// CHAT LOG //
// GET
/api/admin/chat-log – paginated chat history for moderation review
router.get('/chat-log', async (req, res) => {  const roomId =
req.query.roomId as string | undefined;  const page  =
parseInt(req.query.page as string ?? '1', 10);  const limit = 100;  const
messages = await prisma.chatMessage.findMany({  where: roomId ? { roomId }
: {},  include: { sender: { select: { id: true, username: true } } },
orderBy: { createdAt: 'desc' },  skip: (page - 1) * limit,  take:
limit,  });  return res.json(messages); });  //
// SERVER
STATS (ADMIN dashboard overview) //
// GET
/api/admin/stats – high-level alpha metrics (ADMIN only) router.get('/stats',
requireRole(['ADMIN']), async (_req, res) => {  const [  totalUsers,
verifiedUsers, activeUsers,  openReports, totalMessages, invitesUsed,  ]
= await Promise.all([  prisma.user.count(),  prisma.user.count({ where:
{ emailVerified: true } }),  prisma.user.count({ where: { lastLoginAt:
{ gte: new Date(Date.now() - 7 * 86400000) } } }),
prisma.report.count({ where: { status: 'OPEN' } }),
prisma.chatMessage.count(),  prisma.inviteCode.count({ where: { isActive:
false, usedById: { not: null } } }),  ]);  return res.json({  users:
{ total: totalUsers, verified: verifiedUsers, activeThisWeek: activeUsers },
moderation: { openReports },  activity:  { totalChatMessages:
totalMessages },  alpha:  { inviteCodesUsed: invitesUsed },  }); });
export default router;

```

12.2 Admin Route Reference Table

Method	Path	Access	Description
POST	/api/admin/invites/generate	ADMIN	Generate a batch of N invite codes with an expiry date
GET	/api/admin/invites	ADMIN	List all invite codes with usage status
DELETE	/api/admin/invites/:id	ADMIN	Deactivate an unused invite code
GET	/api/admin/users	ADMIN, MOD	Paginated user list with optional username/email search
GET	/api/admin/users/:id	ADMIN, MOD	Single user detail

Method	Path	Access	Description
			with avatar, inventory count, and report count
POST	/api/admin/users/:id/mute	ADMIN, MOD	Mute a player for 1-720 hours
POST	/api/admin/users/:id/ban	ADMIN, MOD	Permanently ban a player
POST	/api/admin/users/:id/unban	ADMIN, MOD	Restore a muted or banned player to ACTIVE
POST	/api/admin/users/:id/promote	ADMIN	Promote a player to MODERATOR role
GET	/api/admin/reports	ADMIN, MOD	Paginated report list — defaults to OPEN; filter with ?status=
PATCH	/api/admin/reports/:id	ADMIN, MOD	Update report status and add moderator notes
GET	/api/admin/chat-log	ADMIN, MOD	Paginated chat history — filter by ?roomId= (100 per page)
GET	/api/admin/stats	ADMIN	High-level alpha metrics: users, open reports, chat volume, invite usage

13 Invite Code Generation and Distribution LINUX

The invite code system gates access to the alpha. Every tester must have a valid, active, unexpired invite code to register. This section covers the CLI generation script, the first-time admin bootstrap, the generation workflow, and the tester distribution template.

13.1 Create the Invite Code CLI Script MAC

Create `apps/server/src/scripts/generate-invites.ts`. This standalone script is run directly on the Oracle VM — not through the API — to generate codes and print them to the terminal. Commit this file from the Mac; run it on Linux.

LINUX-1-G1 — File: `apps/server/src/scripts/generate-invites.ts`

```
import { PrismaClient } from '@prisma/client'; import crypto from 'crypto';
const prisma = new PrismaClient(); // Command-line arguments: // npx ts-
node src/scripts/generate-invites.ts [count] [expiryDays] [adminEmail] const
COUNT = parseInt(process.argv[2] ?? '25', 10); const EXPIRY_DAYS =
parseInt(process.argv[3] ?? '30', 10); const ADMIN_EMAIL = process.argv[4] ??
'dboone9@msn.com'; async function main() { // Verify admin user exists in
the database const admin = await prisma.user.findUnique({ where:
{ email: ADMIN_EMAIL.toLowerCase() }, }); if (!admin)
{ console.error(`\n❌ Admin user with email "${ADMIN_EMAIL}" not
found.`); console.error(' Register your account first at
havenworld.pages.dev, then run:'); console.error(` psql -U havenworld -
d havenworld_prod -h 127.0.0.1`); console.error(` UPDATE users SET role
= 'ADMIN', "emailVerified" = true`); console.error(` WHERE email = '$
${ADMIN_EMAIL}';`); process.exit(1); } if (admin.role !== 'ADMIN') {
console.error(`\n❌ User "${admin.username}" is not an ADMIN (current role: $
${admin.role}).`); console.error(' Promote yourself first with the SQL
command shown above. '); process.exit(1); } const expiresAt = new
Date(Date.now() + EXPIRY_DAYS * 86400000); // Generate unique codes — 8
hex chars = 16^8 = ~4 billion combinations const codes: string[] = [];
while (codes.length < COUNT) { const code =
crypto.randomBytes(4).toString('hex').toUpperCase(); const exists = await
prisma.inviteCode.findUnique({ where: { code } }); if (!exists)
codes.push(code); } await prisma.inviteCode.createMany({ data:
codes.map((code) => ({ code, createdBy: admin.id,
expiresAt, isActive: true, })), }); console.log(`\n✓
HavenWorld Alpha Invite Codes Generated`);
console.log('====='); console.log(`
Count: ${codes.length} codes`); console.log(` Expires: $
${expiresAt.toDateString()} (${EXPIRY_DAYS} days)`); console.log(`
Created: by ${admin.username} (${admin.email})`); console.log(`\n Codes
(share these with alpha testers):`); console.log('
====='); codes.forEach((code, i) =>
console.log(` ${String(i + 1).padStart(2, '0')}. ${code}`));
console.log(`\n Registration URL: https://havenworld.pages.dev`);
console.log(` Instructions: Go to the URL above and click Register.`);
console.log(` Testers enter their invite code during registration.\n`); }
main().catch((err) => { console.error('[Script] Error:', err);
process.exit(1); }) .finally(() => prisma.$disconnect());
```

13.2 First-Time Setup: Promote Yourself to ADMIN LINUX

Before running the invite generation script, your own account must exist in the database with the ADMIN role. This is a one-time bootstrap step performed after your first registration.

LINUX-1-G2 — Step 1: Register your account

Go to **<https://havenworld.pages.dev>**, register using `dboone9@msn.com`, and click the email verification link before proceeding.

LINUX-1-G2 — Step 2: Promote to ADMIN in the database

```
psql -U havenworld -d havenworld_prod -h 127.0.0.1 UPDATE users SET role = 'ADMIN', "emailVerified" = true WHERE email = 'dboone9@msn.com'; SELECT id, username, role FROM users WHERE email = 'dboone9@msn.com'; \q
```

Expected output: one row showing `role = ADMIN`. If the row is not returned, the account does not exist — complete registration first.

13.3 Generate the First Batch of Alpha Invite Codes LINUX

LINUX-1-G3 — Generate 25 codes expiring in 30 days

```
cd /opt/havenworld/apps/server npx ts-node src/scripts/generate-invites.ts 25 30 dboone9@msn.com
```

The script prints all codes to the terminal. **Copy the output to a secure location immediately** — a password manager or private spreadsheet. Codes are also stored in the database and retrievable via `GET /api/admin/invites`.

LINUX-1-G3 — Generate a second wave of codes later

```
# 10 codes expiring in 14 days – for a second wave of testers npx ts-node src/scripts/generate-invites.ts 10 14 dboone9@msn.com
```

13.4 Tester Welcome Email Template

Send each tester the following via Discord DM or direct email. Fill in their name and their unique code. Never reuse codes — one code per tester.

Tester Welcome Message Template

Subject: You're invited to test HavenWorld Alpha!

Hey [Tester Name],

You're officially invited to test HavenWorld — a browser-based social virtual world I'm building.

Your personal invite code: [THEIR_CODE]

How to join:

1. Go to: <https://havenworld.pages.dev>

2. Click "Register"
3. Enter your invite code during signup
4. Check your email for a verification link
5. Log in and explore!

For bugs and feedback: join our Discord at [DISCORD_INVITE_LINK] and post in #bug-reports or #feedback.

Alpha disclaimer: This is pre-release software. Expect bugs. Data may be wiped between major updates — I'll give 48 hours notice in Discord before any wipe.

Thanks for being part of this — your feedback shapes what HavenWorld becomes.

— Daniel

13.5 Track Invite Code Usage LINUX

LINUX-1-G5 — Query invite code status

```
psql -U havenworld -d havenworld_prod -h 127.0.0.1 SELECT ic.code,
ic."isActive", ic."expiresAt", u.username AS "usedBy", ic."usedAt"
FROM invite_codes ic LEFT JOIN users u ON u.id = ic."usedById" ORDER BY
ic."createdAt" DESC; \q
```

14 Production Deployment Workflow

The deployment process is fully automated via GitHub Actions. All code written on the Mac flows to the Oracle VM without any manual SSH steps under normal conditions. This section documents the complete automated flow, the manual emergency path, and the rules for safe database migrations.

14.1 Standard Deployment Flow (Every Code Push)

Step	Where	Action	Details
1	MAC	Write and test code locally	Run <code>pnpm dev</code> — both client and

Step	Where	Action	Details
			server hot-reload on save
2	MAC	Commit and push to develop	git add . && git commit -m "feat: description" && git push origin develop
3	GitHub	PR Check workflow runs automatically	Runs ESLint, TypeScript tsc --noEmit, Jest unit tests, and build compilation. Fix all failures before proceeding.
4	MAC	Open and merge a PR	New Pull Request: develop → main. Wait for green checkmarks. Merge.
5a	GitHub Actions	Client deploy (deploy-client.yml)	Triggers if apps/client/ or packages/shared/ changed. Installs deps → builds Vite client → deploys to Cloudflare Pages. havenworld.pages.dev updates within 30-60 seconds.
5b	GitHub Actions	Server deploy (deploy-server.yml)	Triggers if apps/server/ or packages/shared/ changed. Runs lint + tests as a gate, then SSHs into Oracle VM and executes: git pull → pnpm install → prisma migrate deploy → pnpm build → pm2 restart havenworld-server. Zero manual steps required.

14.2 Manual Emergency Deployment LINUX

Use this path only if GitHub Actions is unavailable or you need to deploy a hotfix immediately without going through a PR.

LINUX-1-H2 — Emergency deployment commands

```
ssh -i ~/.ssh/id_ed25519 ubuntu@[YOUR_ORACLE_IP] sudo -u havenworld -i cd /opt/havenworld git pull origin main pnpm install --frozen-lockfile cd apps/server npx prisma migrate deploy pnpm build pm2 restart havenworld-server pm2 status curl https://[YOUR_IP].nip.io/health
```

14.3 Verify Every Deployment LINUX

Run these checks after every deployment — automated or manual — before considering the deployment complete.

LINUX-1-H3 — Post-deployment verification

```
# 1. PM2 status – havenworld-server must show: online pm2 status # 2. Recent logs – look for "[Server] HavenWorld listening on port 3000" # No ERROR lines should appear in the last 30 lines pm2 logs havenworld-server --lines 30 # 3. Health endpoint – expected: {"status":"ok","timestamp":"...","uptime":...} curl https://[YOUR_IP].nip.io/health
```

Also open **<https://havenworld.pages.dev>** in a Mac browser and confirm the page loads with no console errors.

14.4 Rolling Back a Bad Deployment LINUX

LINUX-1-H4 — Rollback to last known-good commit

```
cd /opt/havenworld git log --oneline -10 # Find the last known-good commit hash, e.g., abc1234 git checkout abc1234 cd apps/server npx prisma migrate deploy pnpm build pm2 restart havenworld-server pm2 logs havenworld-server --lines 20 # Verify server is running correctly before notifying testers git checkout main # Return to main after confirming the rollback works
```

After a successful rollback: fix the issue on Mac, push a new commit to `develop`, open a PR, and merge to `main` to re-deploy normally.

14.5 Rules for Safe Database Migrations MAC LINUX

Data Loss Risk — Read Before Every Schema Change

Alpha player data is real and irreplaceable once testers are playing. A careless

migration can wipe or corrupt it. Follow every rule below without exception.

Rule	Requirement	Wrong	Right
1	Never delete columns in a single migration	Drop column that code still reads	Add new column → migrate data → update code → drop old column in a later migration
2	Never rename columns in one step	ALTER COLUMN rename in one migration	Add new column → copy data → deploy code using new column → drop old column later
3	Test on Mac dev DB first	Edit schema and push directly to production	<code>npx prisma migrate dev --name your-migration-name</code> on Mac first
4	Use <code>migrate deploy</code> on the VM — never <code>migrate dev</code>	<code>prisma migrate dev</code> on Oracle VM	<code>prisma migrate deploy</code> — applies pending migrations safely, no data reset
5	Back up before any destructive migration	Run ALTER/DROP without a backup	<code>pg_dump</code> to <code>/opt/backups/</code> before running the migration

LINUX-1-H5 — Manual database backup before a migration

```
# Create the backup directory once (only needed the first time) sudo mkdir -p /opt/backups
sudo chown havenworld:havenworld /opt/backups # Create a timestamped backup
pg_dump -U havenworld -d havenworld_prod -h 127.0.0.1 \
-F c -f /opt/backups/havenworld_$(date +%Y%m%d_%H%M).dump
```

14.6 Automated Daily Database Backup LINUX

Configure a cron job to back up the database every night at 2:00 AM. The job keeps 7 days of backups and automatically deletes older files — seven dumps of an alpha game are a negligible fraction of the 200 GB Oracle always-free disk allocation.

LINUX-1-H6 — Set up the backup cron job

```
crontab -e # Add the following line (press i to insert, Esc then :wq to save):
0 2 * * * pg_dump -U havenworld -d havenworld_prod -h 127.0.0.1 -F
```

```
c \ -f /opt/backups/havenworld_$(date +%Y%m%d).dump \ && find /opt/backups -name "*.dump" -mtime +7 -delete
```

15 Ongoing Maintenance and Monitoring LINUX

15.1 Daily and Weekly Maintenance Reference

Task	Command	Frequency
Check server status	<code>pm2 status</code>	Every day during alpha
Read recent error logs	<code>pm2 logs havenworld-server --lines 50</code>	After every deployment; each morning
Check disk space	<code>df -h</code>	Weekly
Check RAM usage	<code>free -h</code>	Weekly
Check database size	<code>psql -U havenworld -d havenworld_prod -h 127.0.0.1 -c "SELECT pg_size_pretty(pg_database_size('havenworld_prod'));"</code>	Weekly
View open reports	Call GET /api/admin/reports	After every test session
Check SSL cert expiry	<code>sudo certbot certificates</code>	Monthly
Certificate renewal dry run	<code>sudo certbot renew --dry-run</code>	Monthly

15.2 Useful psql Queries for Alpha Monitoring

Count of registered and verified users

```
SELECT COUNT(*) AS total, COUNT(*) FILTER (WHERE "emailVerified" = true) AS verified FROM users;
```

Active testers — logged in within the last 7 days

```
SELECT username, "lastLoginAt" FROM users WHERE "lastLoginAt" > NOW() - INTERVAL '7 days' ORDER BY "lastLoginAt" DESC;
```

Most active chat rooms in the last 24 hours

```
SELECT r.name, COUNT(cm.id) AS messages FROM chat_messages cm JOIN rooms r ON r.id = cm."roomId" WHERE cm."createdAt" > NOW() - INTERVAL '24 hours' GROUP BY r.name ORDER BY messages DESC;
```

Invite codes still available (unused, not expired)

```
SELECT code, "expiresAt" FROM invite_codes WHERE "isActive" = true AND "usedById" IS NULL AND "expiresAt" > NOW() ORDER BY "expiresAt" ASC;
```

[Users who verified but have never logged in](#)

```
SELECT username, email, "createdAt" FROM users WHERE "lastLoginAt" IS NULL AND "emailVerified" = true ORDER BY "createdAt" DESC;
```

15.3 Handling a Server Crash LINUX

Step 1 — Check PM2 status

```
pm2 status # If havenworld-server shows "errored" or "stopped", proceed to Step 2
```

Step 2 — Read the error log

```
pm2 logs havenworld-server --lines 100 cat /var/log/havenworld/error.log | tail -50
```

Error Message	Cause	Fix
Cannot find module	pnpm install not run after git pull	pnpm install --frozen-lockfile && pnpm build && pm2 restart havenworld-server
ECONNREFUSED redis	Redis is down	sudo systemctl start redis-server && pm2 restart havenworld-server
ECONNREFUSED 5432	PostgreSQL is down	sudo systemctl start postgresql && pm2 restart havenworld-server
Port 3000 already in use	Orphan process on port 3000	sudo fuser -k 3000/tcp && pm2 restart havenworld-server
JWT_ACCESS_SECRET is not set	.env file missing or incomplete	Verify: cat /opt/havenworld/apps/server/.env — check all required keys are present

Step 3 — After fixing, verify recovery

```
pm2 restart havenworld-server pm2 logs havenworld-server --lines 20 curl http://localhost:3000/health
```

15.4 Adding a New Moderator LINUX

When a trusted alpha tester should be given moderation powers, update their role in both the database and the Discord server.

[LINUX — Promote to MODERATOR in database](#)

```
psql -U havenworld -d havenworld_prod -h 127.0.0.1 UPDATE users SET role = 'MODERATOR' WHERE username = 'TrustedTesterUsername'; SELECT id, username, role FROM users WHERE username = 'TrustedTesterUsername'; \q
```

Also assign the **@Moderator** role in Discord server settings so they can see the `#moderator-chat` and `#report-review` channels.

16 Final Alpha-Ready Gate Checklist

Go / No-Go Gate — Do Not Skip Any Item

This is the complete go/no-go checklist before distributing any invite codes. **Every single item must be checked off.** If any item is unchecked, do not hand out invite codes — fix it first. A broken first impression is extremely difficult to recover from in a small alpha.

16.1 Infrastructure Checks

Oracle Cloud VM

VM is in Running state in the Oracle Cloud dashboard

Can SSH into VM from Mac without a password prompt

pm2 status shows havenworld-server as online

curl https://[ip].nip.io/health returns { "status": "ok" }

Nginx is active and HTTPS is working (padlock visible in browser for the nip.io URL)

Let's Encrypt certificate is valid (not expired) — verified with sudo certbot certificates

PostgreSQL is running (sudo systemctl status postgresql shows active)

Redis is running (sudo systemctl status redis-server shows active)

redis-cli -a [password] ping returns PONG

/opt/havenworld/apps/server/.env has all required values — no placeholder strings remaining

PM2 startup is configured (server survives sudo reboot — tested)

/opt/backups/ directory exists and daily backup cron job is active

Server logs show no critical ERROR lines on startup

Cloudflare Pages and GitHub Actions

https://havenworld.pages.dev loads the landing page or game client

Client deploy workflow (deploy-client.yml) ran green on last push to main

Server deploy workflow (deploy-server.yml) ran green on last push to main

PR check workflow (pr-check.yml) ran green on last PR

All GitHub Secrets are populated: CF_API_TOKEN, ORACLE_HOST, ORACLE_SSH_KEY, JWT_ACCESS_SECRET, and all others

VITE_SERVER_URL is set in Cloudflare Pages environment variables and points to the correct nip.io URL

UptimeRobot monitors are active for both havenworld.pages.dev and the API health endpoint

16.2 Game Functionality Checks

Authentication Flow

Registration form works end-to-end: username, email, password, invite code → account created

Verification email arrives in inbox (not spam) and the link works

Login works after email verification

Wrong password returns "Invalid email or password" — no information about which field failed

Unverified email login returns a clear "please verify your email" message

Token refresh fires silently in the background (check network tab — POST /api/auth/refresh fires at ~13 minutes)

Logout clears the session — refresh token is rejected after logout

Invite code is required for registration when ALPHA_INVITE_ONLY=true

Invalid invite codes are rejected with a clear, friendly error message

Core Game Loop — Test With Two Browser Windows Simultaneously

After login, the lobby loads and shows all 4 public rooms with correct occupant counts

Clicking a room card enters the room and loads the tilemap without errors

Local avatar appears in the room at the correct spawn position

Click-to-move works: clicking a walkable tile moves the avatar along a path to the target

Walking animation plays while moving; idle animation plays when stopped

Camera follows the local player with smooth lerp — no jarring jumps

Depth sorting: an avatar walking "below" another renders in front correctly

In Window 2: the Window 1 player's avatar appears and moves in real time

Chat message sent in Window 1 appears in Window 2 within approximately 100ms

Profanity filter replaces filtered words with asterisks

Speech bubble appears above avatar when a chat message is sent

Chat rate limit triggers after 5 messages in 3 seconds with a "Slow down!" message

Entering a different room: avatar leaves the first room (disappears for others) and appears in the new room

Avatar System

Avatar customizer opens from the  Avatar button in the room navigation bar

All inventory items are visible in the correct tabs (tops, bottoms, shoes, hair, etc.)

Clicking an item updates the avatar preview in real time

Saving the avatar persists across logout and re-login

Other players in the room see the avatar update in real time via the avatar:changed event

Personal Room and Furniture

 My Room button navigates to the player's personal room

Furniture items from inventory can be placed in the personal room

Furniture placement persists after leaving the room and returning

Furniture is visible to other players who visit the room

Only the room owner can place or remove furniture — others receive a Forbidden error

Moderation System

Player reporting: right-click or profile card → Report → submits successfully and creates an OPEN report

Admin mute: muted user's chat messages are rejected with "You are temporarily muted"

Admin ban: banned user is disconnected and cannot reconnect while banned

Admin unban: user can reconnect and chat again after unban

Friends System

Friend request can be sent from another player's profile card

Friend request appears in GET /api/friends/requests for the recipient

Accepting a friend request shows both users in each other's friend list

Online status (green dot / grey dot) reflects actual connection state in real time

16.3 Invite Code and Alpha Logistics

Admin account exists and has `role = ADMIN` confirmed in the database Invite code generation script runs successfully: `npx ts-node src/scripts/generate-invites.ts` 25 invite codes have been generated and saved to a secure location Welcome email template is ready and has been tested (sent to yourself first) Discord server is set up with all channels and roles (`#general`, `#announcements`, `#bug-reports`, `#feedback`, `#moderator-chat`) `#bug-reports` channel has pinned instructions for how to report a bug `#feedback` channel has a pinned post with the three key alpha questions (see Section 16.4) Terms of Service and Privacy Policy are linked from the registration page havenworld.hashnode.dev dev blog has at least one published post ROADMAP.md is up to date on GitHub

16.4 The Three Alpha Feedback Questions

Pin these questions in Discord `#feedback` before the first tester logs in. Ask after each test session. These three questions give you 80% of the signal you need to prioritize the next sprint.

 Pin These in #feedback on Day One

Question 1: "Did anything break or not work as expected? Be specific — what were you doing, what happened, and what did you expect to happen?"

Question 2: "How did moving around and chatting feel? Too fast, too slow, awkward, or natural?"

Question 3: "What is the ONE thing you most want to see added next?"

17 Phase 1 → Phase 2 Roadmap Preview

17.1 When to Start Building Phase 2 Features

 **Do NOT build any Phase 2 feature until all three gates are cleared:**

1. At least **5 testers** have logged in and played for at least **30 minutes each**
2. **Day-7 retention** has been measured — at least 3 of 5 testers returned after 7 days
3. The **top 3 bugs** from alpha feedback have been identified and fixed

17.2 Phase 2 Feature Backlog

Feature	Why It Comes After Phase 1	Estimated Complexity
Virtual currency (coins)	Need a stable, validated core loop before introducing an economy. Premature economy mechanics poison the alpha experience.	Medium

Feature	Why It Comes After Phase 1	Estimated Complexity
Player-to-player item trading	Requires currency system + a full anti-exploit audit. Never build before the economy layer is stable and tested.	High
Fishing mini-game	Proven high-engagement feature in comparable games. Add once rooms feel meaningfully alive with real players.	Medium
First seasonal event	Content pipeline must be established first. First event should be planned at minimum 4 weeks in advance.	Medium
Badge and achievement system	Requires real gameplay data to design badges that feel meaningful rather than arbitrary.	Medium
Premium cosmetics / monetization	NEVER before Day-30 retention is confirmed above 20%. Monetizing a product players do not return to is wasted effort and damages trust.	High
Additional public rooms	Expand based on where testers actually congregate — not where you assume they will.	Low
NPC characters	Add only if rooms feel noticeably empty with the real player population present.	Medium
Mobile optimization	Requires a dedicated UX pass. The desktop core loop must be fully solid before splitting attention to mobile.	Medium
Open-source contributor program	Open after the codebase is stable, fully documented, and a CONTRIBUTING.md has been written.	Low
Second language / localization	Phase 3+ feature. Wait until the world identity and content structure are fully established.	High

17.3 The Day-30 Retention Target

The single most important metric before scaling or monetizing HavenWorld is **Day-30 retention above 20%**. This means: of every 10 players who create an account, at least 2 are still playing 30 days later. Measure this before making any significant feature or business decisions.

Query — Calculate Day-30 retention percentage

```
psql -U havenworld -d havenworld_prod -h 127.0.0.1 SELECT COUNT(*) FILTER
( WHERE "createdAt" < NOW() - INTERVAL '30 days' ) AS
"registered30PlusDaysAgo", COUNT(*) FILTER ( WHERE "createdAt" <
NOW() - INTERVAL '30 days' AND "lastLoginAt" > NOW() - INTERVAL '30
days' ) AS "stillActive" FROM users WHERE "emailVerified" = true; \q --
Divide stillActive by registered30PlusDaysAgo × 100 = Day-30 retention %
```

If Day-30 retention is below 20%:

Do NOT add new features. Instead: watch 2-3 testers play (screen share or session recording) and ask "what made you stop playing?" Fix the most common drop-off point in the core loop. Re-measure after 2 more weeks. Repeat until retention exceeds 20% before scaling.

A Appendix — Completed 4-Part Build Guide Summary

All six documents in this series are now complete. The table below maps each part to its key deliverables for quick reference.

Part	Document Title	Key Topics Covered
Part 1	Intro, Free-Tier Stack, Phase 0 Mac	Tech stack decisions, monorepo setup, GitHub repo, cloud accounts, CI/CD workflows, landing page, legal pages
Part 2	Phase 0 Linux — Oracle Cloud	VM provisioning, server hardening, Node.js,

Part	Document Title	Key Topics Covered
		PostgreSQL, Redis, Nginx, Let's Encrypt SSL, PM2
Part 3	Phase 1 Mac — Client, Phaser, Avatar, Maps	Vite + Phaser 3 setup, scene architecture, auth/socket services, HTML overlay, avatar sprites with LibreSprite, Tiled room maps, avatar customizer, click-to-move pathfinding
Part 4A	Phase 1 Linux — Server Bootstrap and DB	Server entry point, Prisma schema (all 10 models), migrations, database seeding, auth middleware
Part 4B	Phase 1 Linux — Auth, Sockets, Services	Full auth REST API, Socket.io event handlers (auth:join, player:move, chat:send, avatar:update, disconnect), RoomManager, ModerationService
Part 4C	Phase 1 Linux — Admin, Deploy, Checklist (<i>This Document</i>)	Admin moderation routes, invite code generation CLI, production deployment workflow, maintenance commands, 60+ item alpha-ready gate checklist, Phase 2 roadmap preview

Final Note — Series Complete

All six documents in this series — Parts 1, 2, 3, 4A, 4B, and 4C — are now complete. When every item in the Section 16 checklist is checked off, HavenWorld is ready for its first alpha testers.

The single most important thing to do after the first testers log in: **watch them play without helping.** Observe what confuses them, what delights them, and what makes them stop. That one observation session is worth more than any feature you could add in the same amount of time.

Build the world. Invite the players. Listen to them.

Good luck, Daniel. — HavenWorld Build Guide, September 18, 2026